

Low-Code Programming Models

Focus on Programming by Demonstration and Programming by Example

Tianyiming Jing

University of Michigan, CSE 590

Spring 2026

Outline

- 1 Introduction
- 2 Hirzel 2023 – Taxonomy
- 3 Sutherland 1963 – Sketchpad
- 4 Cypher 1991 – EAGER
- 5 Lau et al. 2003 – Version Space Algebra
- 6 Gulwani 2011 – FlashFill (In-Depth)
- 7 Polozov & Gulwani 2015 – FlashMeta
- 8 Open Research Questions
- 9 Conclusion

The Problem

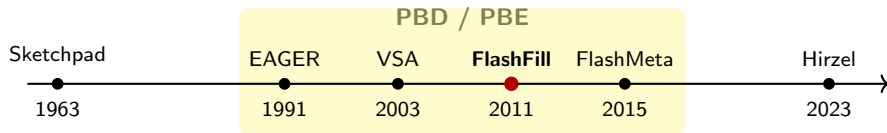
- Software is everywhere, but only a small fraction of people can create it
- Traditional programming requires learning syntax, semantics, and tools
- Many users have domain expertise but no programming background
- **Goal:** let users create programs without writing conventional code

Low-Code Approaches

- **Visual programming:** users build programs by arranging graphical elements (blocks, wires, diagrams)
- **Programming by demonstration (PBD):** user performs a task, system watches and learns to automate it
- **Programming by example (PBE):** user provides input-output pairs, system synthesizes a program
- **Natural-language programming:** user describes what they want in English, system generates code (e.g., Codex, Copilot) – context only, not covered today

This presentation focuses on **PBD and PBE**, with brief coverage of visual programming as the ancestor.

Roadmap



- 1 Hirzel 2023 – Taxonomy (survey, big picture)
- 2 Sutherland 1963 – Sketchpad (visual programming)
- 3 Cypher 1991 – EAGER (early PBD)
- 4 Lau et al. 2003 – Version Space Algebra (PBD theory)
- 5 **Gulwani 2011 – FlashFill (PBE, in-depth)**
- 6 Polozov & Gulwani 2015 – FlashMeta (PBE framework)

Low-Code Taxonomy (Hirzel, 2023)

Problem: Visual, demonstrative, and natural-language programming have been studied as separate fields with little interaction.

Solution: A side-by-side comparison of three low-code techniques.

Technique	Ambiguity	Need to learn
Visual prog. langs. (VPL)	low	medium
Prog. by demonstration (PBD)	medium	low
Prog. by natural lang. (PBNL)	high	low

Common backbone: domain-specific languages + model-view-controller.

Hirzel's Three Techniques (Figs. 2–4)

Figure 2. Visual programming languages.

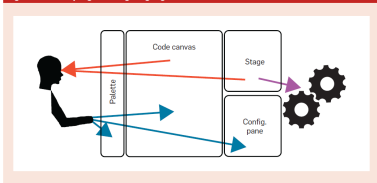


Fig. 2: VPL

Figure 3. Programming by demonstration.

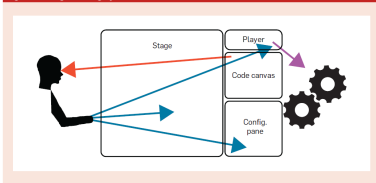


Fig. 3: PBD

Figure 4. Programming by natural language.

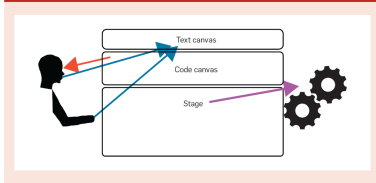


Fig. 4: PBNL

Same building blocks (code canvas, stage, palette, config pane, ...) rearranged: **red arrows** = read, **blue arrows** = write, **purple arrows** = execute.

Evaluation: qualitative survey, illustrated with concrete systems (Scratch, EAGER, FlashFill, Pumice, Rousillon).

Limitations: inherits its sources' limitations; doesn't yet capture hybrid systems.

Sketchpad (Sutherland, 1963)

Problem: In the 1960s, all programming was textual and batch-oriented. Non-programmers had no way to interact with computers.

Solution: Sketchpad, a graphical editor for the TX-2 computer.

- Users draw shapes with a light pen and define constraints (parallel, equal length, ...)
- System satisfies constraints via a *relaxation* method
- Users define “master” drawings and create “instances” as subpictures (anticipates OOP)

Evaluation: Demonstrated through example drawings: mechanical linkages, bridges (Figure 15), circuit diagrams.

Limitations: the relaxation method is “unacceptably slow” for many instances; required the specialized TX-2 and light pen.

Sketchpad – Figures 2 and 3

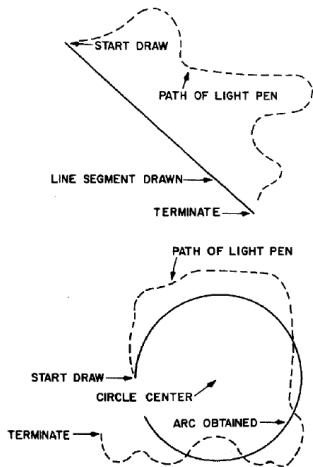


Figure 2. Steps for drawing straight lines and circle arcs.

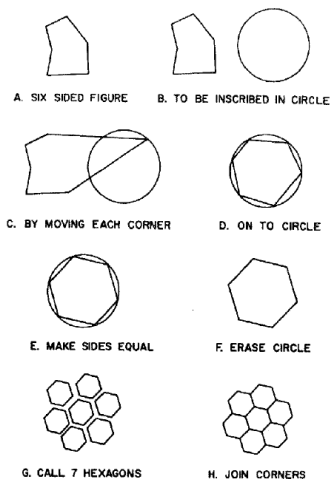


Figure 3. Illustrative example, see text.

EAGER (Cypher, 1991)

Problem: Macro recorders required users to explicitly start/stop recording, and could not generalize beyond the exact recorded actions.

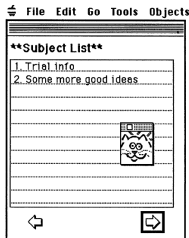
Solution: EAGER, a PBD agent for HyperCard that runs constantly in the background.

- Watches the user work and never asks for information
- Uses a new interface technique called *anticipation*: when it detects a repeating pattern, it highlights the next predicted menu item, button, or text selection in green
- When the user is confident, they click the Eager icon and EAGER writes a HyperTalk program that completes the remaining iterations

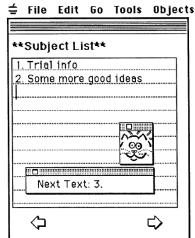
Evaluation: User study with 7 first-time subjects on 3 repetitive tasks. Subjects were given no information about EAGER. The system detected patterns in 17 of 21 task instances.

Limitations: cannot handle conditionals, nested loops, or repetitions spread out over time; anticipation cannot communicate termination conditions; tied to HyperCard.

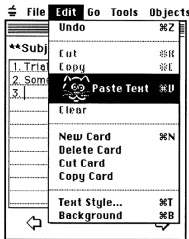
EAGER – Anticipation in Action (Fig. 1)



(e)



(e)



(f)



(g)

Version Space Algebra (Lau et al., 2003)

Problem: Earlier PBD systems (like EAGER) relied on ad hoc heuristics or brute-force search. Both scaled poorly.

Solution: A formal framework called *version space algebra*.

- Extends Mitchell's version spaces from concept learning to arbitrary functions
- Compactly represents **all hypotheses** consistent with the demonstrations
- Composes simple version spaces into complex ones via algebraic operators (*union* and *join*)
- Each new demonstration refines the space incrementally

Evaluation: SMARTedit tested on 30 text-editing scenarios (Table 2); many learnable from 1–2 iterations. Pilot user study with 6 undergraduates on 7 tasks.

Limitations: cannot learn loop termination conditions on its own; restricted to text-editing; some scenarios needed many iterations due to data irregularities.

FlashFill (Gulwani, 2011)

Problem: Millions of spreadsheet users need to reformat, extract, or combine text data, but cannot write the formulas or scripts needed.

Approach: Programming by example (PBE)

- User provides a few input-output examples in a spreadsheet column
- System synthesizes a string-processing program that produces the correct outputs
- Applied to the rest of the column automatically

	A	B	C
1	First Name	Last Name	Result
2	Tony	Stark	Stark, Tony
3	Clark	Kent	Kent, Clark
4	Peter	Parker	Parker, Peter
5	Steve	Rogers	Rogers, Steve
6	Bruce	Wayne	Wayne, Bruce
7	Harry	Potter	Potter, Harry
8	Albus	Dumbledore	Dumbledore, Albus

Source: trumpexcel.com

FlashFill – Step 1: Define a DSL

Define a domain-specific language of string expressions (Fig. 1 from the paper):

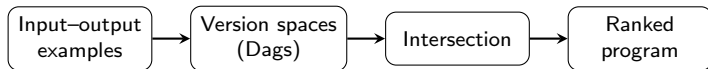
$$\begin{aligned}\text{String expr } P &:= \text{Switch}((b_1, e_1), \dots, (b_n, e_n)) \\ \text{Bool } b &:= d_1 \vee \dots \vee d_n \\ \text{Conjunct } d &:= \pi_1 \wedge \dots \wedge \pi_n \\ \text{Predicate } \pi &:= \text{Match}(v_i, r, k) \mid \neg \text{Match}(v_i, r, k) \\ \text{Trace expr } e &:= \text{Concatenate}(f_1, \dots, f_n) \\ \text{Atomic expr } f &:= \text{SubStr}(v_i, p_1, p_2) \\ &\quad \mid \text{ConstStr}(s) \\ &\quad \mid \text{Loop}(\lambda w : e) \\ \text{Position } p &:= \text{CPos}(k) \mid \text{Pos}(r_1, r_2, c) \\ \text{Integer expr } c &:= k \mid k_1 w + k_2 \\ \text{Regular Expression } r &:= \text{TokenSeq}(T_1, \dots, T_m) \\ \text{Token } T &:= C + \mid [\neg C] + \\ &\quad \mid \text{SpecialToken}\end{aligned}$$

Figure 1. Syntax of String Expressions P . v_i refers to a free string variable, while w refers to a bound integer variable. k denotes an integer constant and s denotes a string constant.

Full denotational semantics given in Fig. 2 of the paper; omitted here for space.

Synthesize via version-space intersection

- For each input-output example, build a data structure that compactly represents *all* DSL programs consistent with it
- Intersect these representations across all examples
- Rank the remaining programs and pick the most likely one



FlashFill – Example

Extract the quantity at the end of each item description:

Input v_1	Output
BTR KRNL WK CORN 15Z	15Z
CAMP DRY DBL NDL 3.6 OZ	3.6 OZ
CHORE BOY HD SC SPNG 1 PK	1 PK
FRENCH WORCESTERSHIRE 5 Z	5 Z

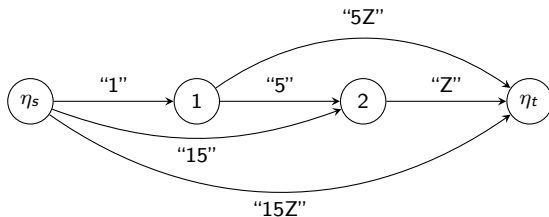
From the first example, FlashFill synthesizes:

$SubStr(v_1, Pos(\epsilon, NumTok, 1), CPos(-1))$

“Take the substring from just before the first number to the end of the string.”

FlashFill – Inside the Dag

How can we represent *all* programs consistent with one example? A **Dag** where nodes are output positions and edges carry sets of atomic expressions. For output “15Z”:



- **Each edge carries a set** of atomic expressions (many SubStrs with different Pos, plus a ConstStr)
- **Any source-to-target path** = a valid Concatenate program
- Compactly encodes **exponentially many** consistent programs
- **Intersecting two Dags** is analogous to intersecting two regular automata – node pairs, edge pairs, recursive set intersection

FlashFill – Evaluation

- Tested on **more than 100 string-processing benchmarks** drawn from Excel online help forums and the Excel product team
- Each problem provided up to 10 input-output pairs (more than a typical user would supply)
- Algorithm runs in **less than 0.1 seconds on average** per interactive round
- Typically converges in **1–4 rounds** of interaction

(Post-publication: the technique was later integrated into Microsoft Excel 2013 as the “Flash Fill” feature.)

FlashFill – Limitations

- The synthesis engine **does not support arithmetic reasoning**. Example 14 (summing numbers in a string) had to use a workaround.
- Restricted form of regular expressions: only one or more occurrences (not zero or more) and no disjunction
- Tokens are predefined; problem-specific tokens must be added manually

More broadly, when a few examples leave many DSL programs consistent, the system has to rank and pick one, and the chosen program may not match the user's intent.

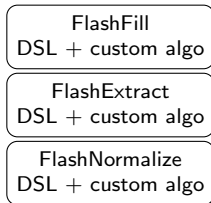
FlashMeta (Polozov & Gulwani, 2015)

Problem: Building a new PBE system for each domain (strings, tables, web pages) required writing a new DSL and synthesis algorithm from scratch each time.

Solution: FlashMeta, a general framework for building PBE systems.

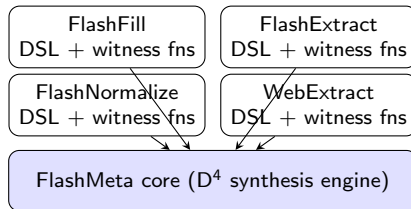
- Developer defines a DSL and provides *witness functions*
- Witness functions describe how to propagate examples through each DSL operator
- FlashMeta automatically derives the synthesis algorithm

Before FlashMeta



~1 year per system

With FlashMeta



~1 month per system

FlashMeta – Evaluation and Limitations

Evaluation: Industrial collaborators reimplemented 4 prior systems and built a new one (WebExtract). From Table 2 of the paper:

Project	Lines of code		Development time	
	Original	FlashMeta	Original	FlashMeta
FlashFill (Gulwani 2011)	12K	3K	9 months	1 month
FlashExtract (Le & Gulwani)	7K	4K	8 months	1 month
FlashNormalize (Kini & Gulwani)	17K	2K	7 months	2 months
FlashRelate (Barowy et al.)	5K	2K	8 months	1 month

Limitations: D^4 requires the DSL to be characterized by decidable witness functions; witness functions must still be written manually for each operator; best suited for DSLs with rich semantics, less so for those already covered by SMT solvers.

Open Research Questions

- ❶ **Correctness and trust:** FlashFill can synthesize the wrong program from ambiguous examples; users have no easy way to verify synthesized programs.
- ❷ **Expert domains:** most PBD/PBE targets spreadsheets and text editing. Can it work for formal verification, scientific computing, or hardware design? (E.g., PGSynth applies PBD to microarchitectural verification.)
- ❸ **Combining modalities:** users might want to draw a diagram, provide examples, and then refine with natural language. How do we design smooth transitions between modalities?
- ❹ **Scalability:** current PBD/PBE works well for small, isolated tasks. Can it scale to larger, evolving software systems?

- Low-code programming has evolved over six decades
- PBD/PBE went from ad hoc heuristics (EAGER) to formal frameworks (version spaces) to industrial tools (FlashFill, FlashMeta)
- Visual programming (Sketchpad) is the ancestor; Hirzel's taxonomy situates PBD/PBE alongside it and natural-language programming
- Key open problems: correctness/trust, expert domains, combining modalities, scalability

Thank you! Questions?